

ЛР №0. Техническое задание: CO-RAG — RAG-ассистент по архиву журнала «Компьютерная оптика»

Курс «Технологии сетевого программирования» · Исполнитель: Кравченко Иван Антонович, гр. 6402 (индивидуально) ·
Репозиторий: будет добавлен после ЛР №1

1. Введение

CO-RAG переводит сканы выпусков журнала «Компьютерная оптика» в текст и отвечает на вопросы по архиву со ссылками на статью, выпуск и страницу. Часть старых выпусков существует только в виде сканов; статьи содержат формулы и таблицы на русском и английском языках.

Задачи ИС: OCR сканов (DeepSeek-OCR → Markdown); хранение структуры архива (выпуски, страницы, статьи, авторы, ключевые слова); индексация (чанки + эмбединги Jina в PostgreSQL/pgvector); ответы по схеме наивного RAG с локальной LLM; разграничение доступа двух ролей (JWT); уведомления о ходе обработки в реальном времени (RabbitMQ + WebSocket). **Вне рамок:** история диалогов, ручная правка OCR, веб-интерфейс, реранкинг.

2. Бизнес-цели и функционал

№	Цель (что?)	Метрика и цель	Функции (как?)
B1	Сократить время поиска ответа по архиву	≤ 30 с до ответа с источниками (p95)	Ф4, Ф5
B2	Перевести сканы архива в машиночитаемый текст	≥ 95 % страниц загруженных выпусков с текстом	A1–A3
B3	Сделать ответы проверяемыми	100 % ответов со ссылкой на статью и страницу, иначе явный отказ	Ф3, Ф4
B4	Сделать обработку прозрачной для администратора	≤ 5 с от события до уведомления, без опроса	A4, Ф6

Исследователь (researcher, по умолчанию при регистрации)

- Ф1 Регистрация и вход, получение JWT
- Ф2 Каталог выпусков и статей: фильтр по году, автору, ключевому слову
- Ф3 Карточка статьи и текст её страниц
- Ф4 Вопрос к архиву → ответ LLM + источники
- Ф5 Семантический поиск фрагментов без генерации
- Ф6 Уведомление о новом выпуске (WebSocket)

Администратор (admin, наследует права исследователя)

- A1 CRUD выпусков, загрузка PDF-скана
- A2 Запуск OCR выпуска (асинхронная задача)
- A3 Автоматическая индексация страниц после OCR
- A4 Прогресс и ошибки OCR в реальном времени (WebSocket)
- A5 CRUD статей (диапазон страниц), авторов, ключевых слов
- A6 Управление пользователями: роль, блокировка

3. Сценарии использования

- Вопрос к архиву.** POST /auth/login → JWT → POST /rag/ask {question, year_from, year_to} → JSON: ответ + citations (статья, выпуск, страница, фрагмент) → GET /articles/{id}. Нет релевантных фрагментов → «не найдено» без вызова LLM; нет токена → 401.
- Загрузка и OCR выпуска.** Админ подключает WS-клиент (/ws?token=...) → POST /issues → POST /issues/{id}/scan (PDF) → POST /issues/{id}/ocr → 202 + job_id → по WebSocket приходят ocr.started, ocr.progress, ocr.completed, issue.indexed. Повторный запуск при активной задаче → 409; роль researcher → 403.
- Разметка статей.** GET /issues/{id}/pages/{n} → POST /authors → POST /articles {issue_id, title, page_start, page_end, author_ids, keywords} → правка PATCH, удаление DELETE. После этого ответы RAG по страницам статьи ссылаются на неё.

4. Архитектура

Схема повторяет архитектуру курса: **API Service** (FastAPI, модули auth и db) ↔ **RAG Service** на gRPC; RabbitMQ → **Notification Service** (WebSocket) → клиент. Добавлены OCR Worker (GPU), LLM Server (Ollama/vLLM) и pgvector.

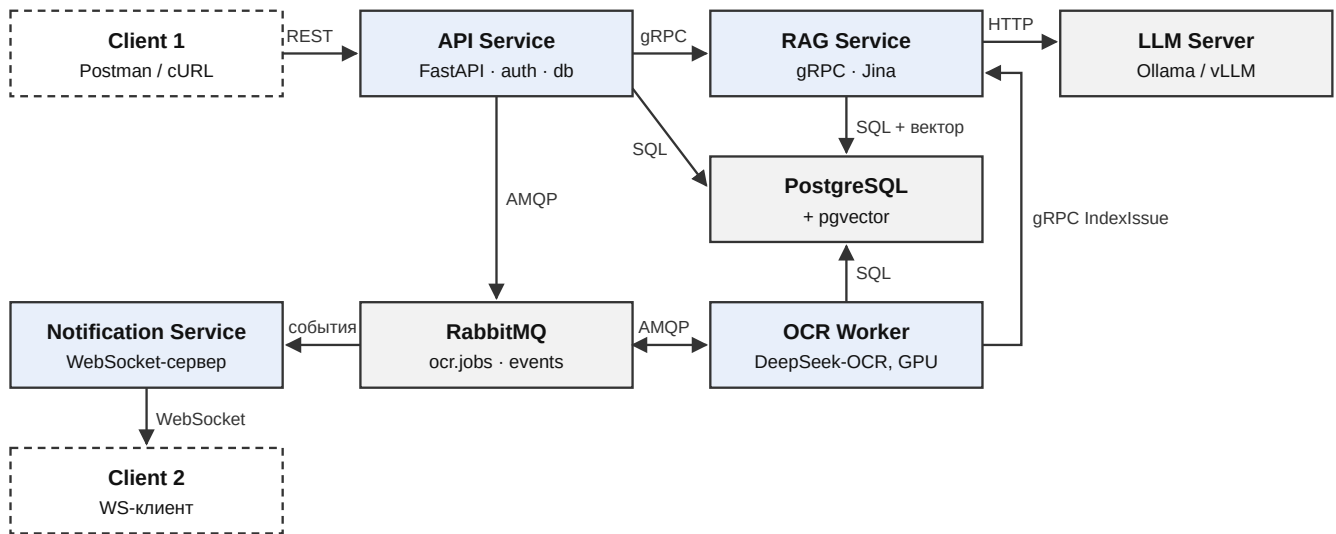


Рис. 1. Диаграмма компонентов (пунктир — клиенты, голубые — сервисы проекта, серые — инфраструктура)

Компонент	Роль
API Service	REST API (FastAPI + Pydantic DTO), JWT-аутентификация, SQLAlchemy-репозитории через DI; вызывает RAG по gRPC, публикует задачи OCR в RabbitMQ
RAG Service	gRPC-методы Ask, Search, IndexIssue (Protobuf): эмбединг Jina → top-k чанков в pgvector → ответ LLM
OCR Worker	Берёт задачу из очереди ocr.jobs, распознаёт страницы в Markdown, пишет их в БД, публикует события, вызывает IndexIssue
Notification Service	Читает события из exchange events, проверяет JWT при подключении, рассылает JSON по WebSocket
Инфраструктура	PostgreSQL 16 + pgvector, RabbitMQ, LLM Server; всё запускается docker compose up, Nginx — опционально

Форматы: JSON — REST, сообщения RabbitMQ и WebSocket; Protocol Buffers — gRPC.

5. Структура базы данных

8 сущностей, 10 таблиц. Связи M:N: **статья** ↔ **автор** (article_authors, с порядком автора) и **статья** ↔ **ключевое слово** (article_keywords).

Таблица	Основные поля (PK подчеркнут, → FK)
users	<u>id</u> , email UNIQUE, password_hash, full_name, role {researcher, admin}, is_active
issues	<u>id</u> , year, volume, number, scan_path, page_count, status, created_by → users; UNIQUE(year, number)
pages	<u>id</u> , issue_id → issues, page_number, text_md; UNIQUE(issue_id, page_number)
articles	<u>id</u> , issue_id → issues, title, abstract, doi, language, page_start, page_end
authors / keywords	<u>id</u> , full_name, affiliation, orcid / <u>id</u> , value UNIQUE
article_authors	<u>article_id</u> → articles, <u>author_id</u> → authors, author_order
article_keywords	<u>article_id</u> → articles, <u>keyword_id</u> → keywords
chunks	<u>id</u> , page_id → pages, chunk_index, text, embedding VECTOR(1024); индекс HNSW (cosine)
ocr_jobs	<u>id</u> , issue_id → issues, requested_by → users, status, pages_total, pages_done, error

Статья связана со страницами диапазоном page_start..page_end, поэтому индексация не ждёт разметки статей.

6. Соответствие формальным требованиям

- Чтение, запись, редактирование БД — CRUD выпусков, статей, авторов, ключевых слов, пользователей; запись страниц, чанков, задач OCR.
- Две роли — researcher и admin, проверка роли из JWT.
- ≥ 3 сущностей и связь M:N — 8 сущностей, две связи M:N.